AI-Enabled Multi-Task Robot Work-Cell

Master’s capstone | Santa Clara University | Final report: Summer 2025

Project status: Integrated desktop research workcell with documented tool-detection and pick-and-place experiments. The system uses Python, stereo perception, a trained detector, and serial robot commands; this implementation is separate from the author’s later ROS 2 DexArm package.

1) Project overview

This project connects visual perception to robotic action in a tool-handling workcell. A ZED stereo camera observes tools on a conveyor, YOLOv8 classifies them, and depth/image processing estimates attributes needed for manipulation. A Rotrics DexArm receives calibrated position and end-effector orientation targets, attempts a grasp, and transfers the tool to a placement location.

The demonstrated classes are Hammer, Wrench, and Screwdriver. The work goes beyond label detection by integrating camera-to-robot coordinate mapping, size and color characterization, orientation alignment, separate monitoring and actuation clients, and visual pickup checks. It explores how those subsystems affect practical grasp performance.

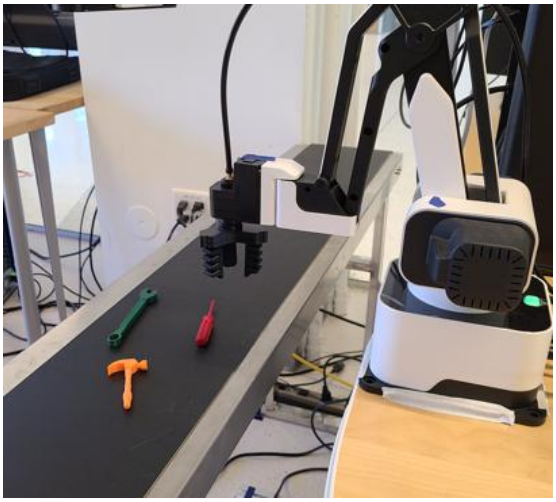


Figure 1. Physical workcell and tool scene. Source: capstone report, Figure 5.2.

Individual contribution

The author confirms responsibility for all submitted scripts and the development described in the report: perception integration, calibration, communication, robot control, testing, and documentation. The report acknowledges academic supervisors and teammates; the individual implementation attribution here follows the author’s clarification.

2) Hardware and software architecture

Component	Role
ZED stereo camera	Left-image capture and per-pixel XYZ measurements via the ZED SDK.
Rotrics DexArm	Cartesian positioning, rotary end-effector alignment, and soft-gripper pick/place commands.
Conveyor and support structure	Tool presentation and spatial arrangement of the camera and robot.
Host computer	Runs the vision server and two Python client processes.
Lightweight tools	The report describes 3D-printed tools used within arm/payload and gripping constraints.

Software responsibilities

File	Responsibility
sender4.py	Detection, attributes, target mapping, data sockets, and verification service.
s1client4.py	Receives descriptive tool messages and displays them in the terminal.
s2client4.py	Receives robot targets, manages command queue and arm motion, and requests pickup verification.

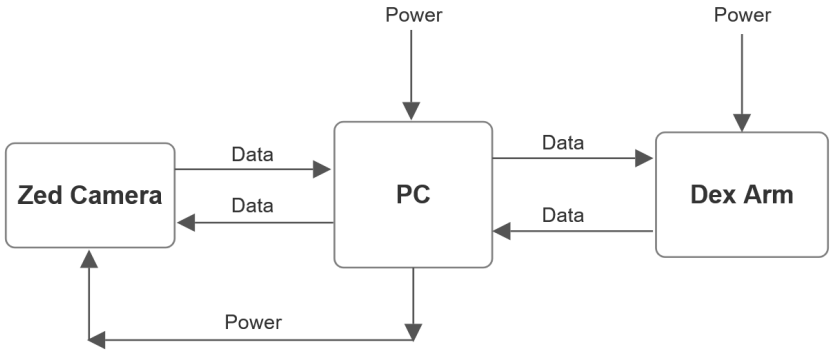


Figure 2. Author-supplied component diagram. The implemented interprocess communication uses local TCP sockets.

The architecture separates high-rate visual observation from robot movement and human-readable monitoring. It is modular at process level: the monitoring client does not command the arm, while the robot client can act on target data without owning the detector. The supplied server accepts up to two clients on 127.0.0.1:65432; verification requests use port 65433.

High-level software flowchart

Supplement to Section 2: Hardware and software architecture

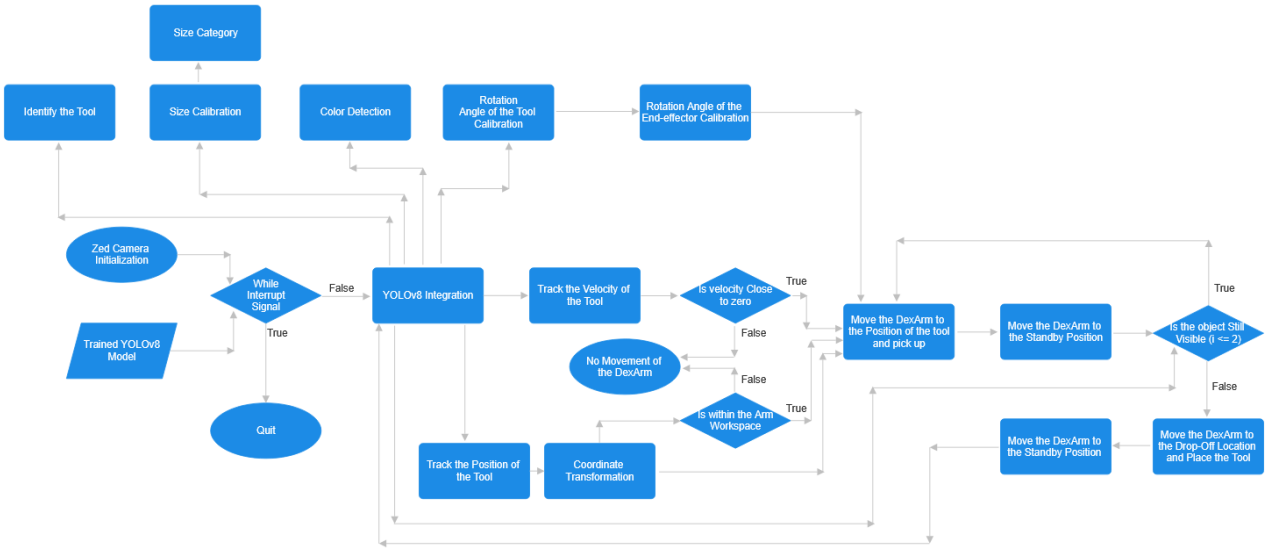


Figure 2A. Author-supplied High Level Software Flowchart.png, rotated for upright reading. Source: project ZIP, Diagrams folder.

Workflow overview

The diagram connects ZED camera initialization and the trained YOLOv8 model to tool identification, position tracking, size and color processing, and orientation calibration. The manipulation branch relates target coordinates and end-effector alignment to pickup, standby positioning, visual checking, and placement.

Design and implementation distinction

The chart documents the intended high-level sequence. The submitted code calculates velocity status but does not enforce it before transmitting robot targets. Its grasp-retry loop also reuses the original received coordinates and angle. The detailed explanations in Sections 3-7 describe the behavior established by the submitted scripts.

3) Detection and 3D position estimation

sender4.py loads my_model1.pt through the Ultralytics YOLO API. Camera settings select HD720 resolution, ULTRA depth mode, and millimeter coordinate units. Each successful camera grab retrieves the left image and XYZ data; the BGRA image is converted to RGB before inference.

The active code retains only Hammer, Wrench, and Screwdriver predictions at confidence 0.6 or above. It applies scene-specific filters for very large boxes and detections near the lower image boundary. An additional overlap pass uses intersection-over-union above 0.5 and retains the more confident overlapping detection. These are configured processing rules, not measured classification accuracy.

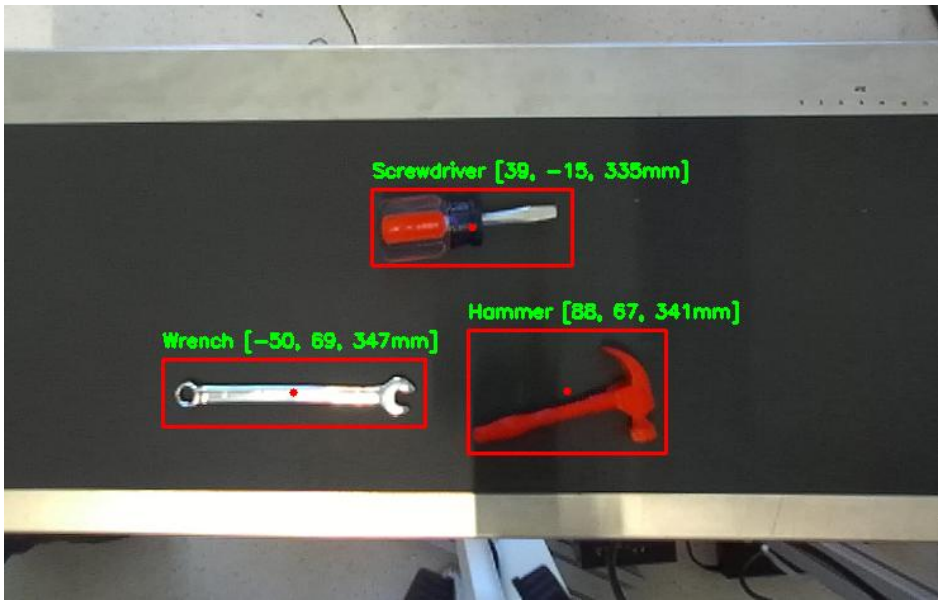


Figure 3. Tool classes, 2D detection boxes, and camera-frame coordinates. Source: report Figure 3.3.

Position and motion data

The center pixel of each accepted box is used to obtain XYZ coordinates. Invalid depth samples are skipped. The code also computes position differences over time for velocity status. However, object identity is based on Python object identity rather than a demonstrated persistent multi-object tracker; velocity gating is not enforced before transmitting robot targets in this snapshot.

The report discusses stationary or nearly stationary picking. The supplied final scripts do not demonstrate conveyor start/stop commands or continuous moving-target interception. The project is therefore described as vision-guided handling in a conveyor workcell rather than guaranteed on-the-fly picking.

4) Color, size, and orientation

Color classification

The report describes HSV conversion, color masks, morphological processing, and pixel-count decisions within a detection region. The active server calls color_det1 from ColorDetection1.py. That module is absent from the ZIP, so the exact active thresholds cannot be reproduced from the submitted files alone. Black-object classification is identified in the report as a limitation of the dark background.

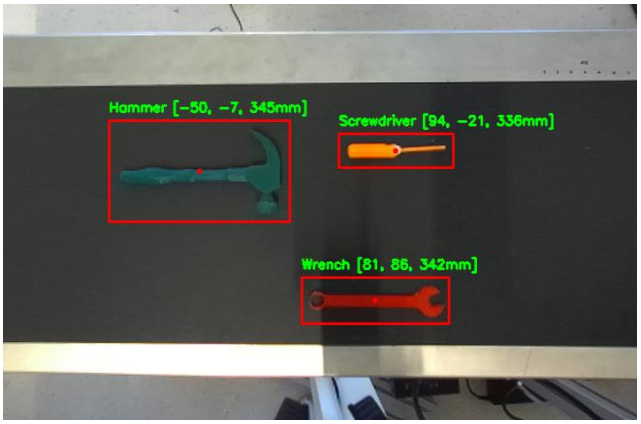


Figure 4. Different tool colors in the camera view. Source: report Figure 3.4.

Depth-derived size estimates

The server samples a grid of points inside each detection at a stride of three pixels, rejects invalid XYZ values, removes percentile outliers on each axis, and estimates width, height, and depth from coordinate extrema. The maximum estimated dimension determines a Small, Medium, or Large category. Active threshold constants are lower_thres=83, upper_thres=124, and noise_val=8, in the camera's millimeter units.

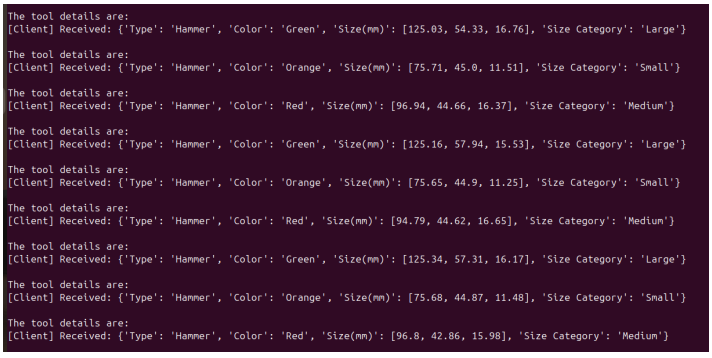


Figure 5. Size/color messages for hammers of different sizes. Source: report Figure 3.6. These are depth-derived estimates, not certified dimensional measurements.

End-effector alignment

Orientation processing converts the region to grayscale, blurs it, extracts Canny edges, and fits a minimum-area rectangle to the largest contour. The rectangle angle is adjusted by its width/height relationship and offset by 15 degrees for the end-effector command. If no contour is available, the command falls back to 15 degrees.

5) Calibration and communication

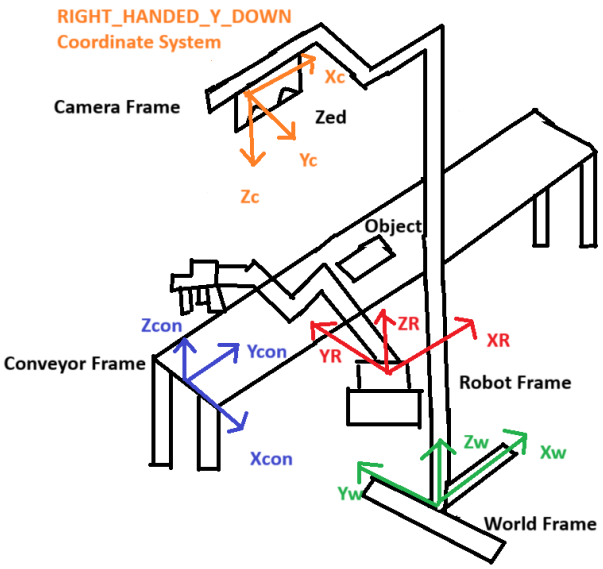


Figure 6. Workcell coordinate frames supplied by the author.

Implemented target mapping

The final server maps the measured camera point using rx = x - 35 and ry = 325 - y, and uses a fixed rz = -117 for the configured picking plane. These are setup-specific offsets/sign conventions rather than a general calibrated rigid-body transform. The end-effector angle is transmitted separately as ea.

Message group	Fields used in the submitted implementation
Monitoring	Type, Color, Size(mm), Size Category.
Robot target	rx, ry, rz, cx, cy, cz, ea, label.
Verification request	cmd="verify", label, x, y, z.
Verification response	success plus a status message or error.

Messages are JSON objects terminated by newline characters. Both clients accumulate TCP bytes in a buffer and split complete lines before decoding, allowing messages to arrive in fragments. The server assigns the first connected client to descriptive data and the second to robot targets; startup order matters. Start sender4.py, then s1client4.py, then s2client4.py in separate terminals for the intended roles.

The code suppresses transmission when a message matches the previously transmitted data. This reduces duplicate output but does not establish synchronized sensing/control deadlines. It is local interprocess communication, not a network-distributed real-time guarantee.

6) Robot execution and grasp verification

s2client4.py connects to the arm through /dev/ttyACM0 using the DexArm Python API. A socket-listener thread receives targets while a worker consumes a one-slot command queue. An arm_busy flag discards incoming targets during motion; older queued targets are cleared before a new command is queued.

The workspace check accepts a radial distance between 245 and 360 mm and checks transformed depth against a configured vertical band. It then rotates the end-effector, moves to the target, actuates the soft gripper, and returns to a standby pose. After verification succeeds, the arm moves to the drop-off location, releases, resets the gripper, and returns.

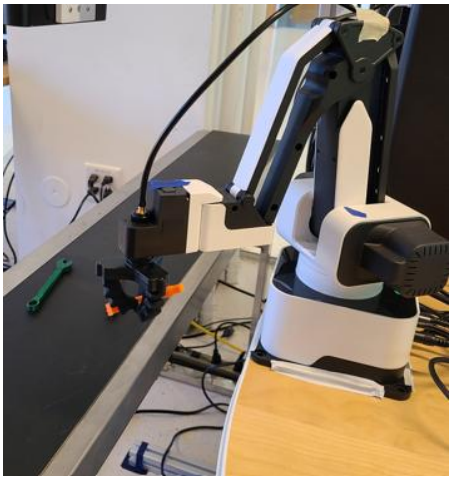


Figure 7. Pickup sequence from the report, Figure 5.4.

Visual verification logic

After the robot moves away, the server captures another view and looks for a same-label detection of confidence at least 0.6 whose XYZ coordinates remain within 35 mm on every axis of the original point. Finding one means the target is still present and another attempt is requested; not finding one is interpreted as pickup success.

This is indirect visual confirmation: disappearance may also result from occlusion, missed detection, or object movement. The code uses max_retries=2 with a loop condition attempt <= max_retries, allowing up to three total attempts. All attempts for that queue item reuse its original target coordinates and angle; the worker does not refresh them from verification output.

7) Experimental evaluation

Chapter 6 of the report compares three configurations, with ten trials per configuration and three tools presented per trial. Each trial records attempts and successful pickups. The reported metric is the average of the per-trial ratios successes/attempts, not detector accuracy or percentage of objects ultimately retrieved.

Configuration as described in report	Reported average
Fixed end-effector angle	30.2%
End-effector angle aligned to tool orientation	70.8%
Orientation plus position/orientation feedback updates (Test 3)	78.5%

The difference between the first and third reported averages is 48.3 percentage points. These are the report's recorded figures; they are not new measurements or an independent reproduction. Trials include retries, so this metric should not be presented as first-attempt grasp success.

Report/code distinction: Test 3 is described as updating the target after each grasp check, whereas the submitted client reuses its original received target. The author confirms these are the final working scripts. The figures are retained as report-recorded results without attributing refreshed-target behavior to that code.

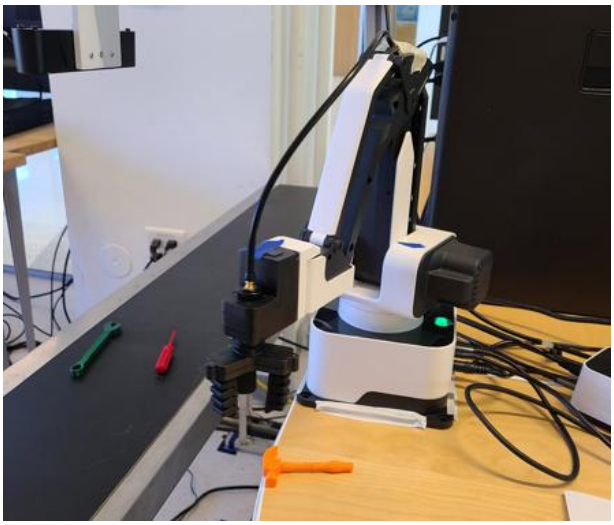


Figure 8. Transfer and placement sequence. Source: report Figure 5.5.

The results support an engineering discussion of orientation alignment and feedback-assisted handling in the tested workcell. They do not establish performance across different lighting, tool materials, camera mounting, conveyor speeds, or industrial production loads.

8) Limitations, future work, and sources

Current limitations

Lighting and shadows influence detection, contour orientation, and color results. The dark background complicates black-tool color identification. Soft-gripper stability and arm payload constrain tool selection. Fixed plane/offset calibration limits transferability to another geometry. Perception and verification share camera/model resources across threads without a fully demonstrated synchronized acquisition design.

Recommended next steps

Refresh target coordinates and orientation before retries; add gripper or force/vacuum confirmation; measure camera/robot extrinsics; stabilize illumination; evaluate persistent object tracking; and validate commands and workspace transitions. Record cycle time, localization error, first-attempt success, overall retrieval rate, and recovery behavior separately.

Conclusion

The project demonstrates end-to-end integration of learned object detection, stereo measurements, classical image processing, local communication, and robot manipulation. The strongest contribution is the coordinated system implementation and experimental exploration of grasping behavior, with clear opportunities for stronger calibration and validation.

Source register

AI-Enabled Multi-Task Robot Work-Cell.zip: Capstone Report Final Dak Amarasinghe.pdf (52 pages; Figures 2.1-2.6, 3.3-3.7, Chapter 5 execution sequences, and Chapter 6 tests); sender4.py; s1client4.py; s2client4.py; Instructions.txt; and five diagram PNGs. Résumé: Dak Amarasinghe Resume(20261030-094031).pdf. Individual contribution and unused-import clarification supplied by the author.

Prepared by reviewing source, documentation, and images; no new physical workcell experiment was run. General technology discussions in the report (such as TensorRT/ONNX) are not treated as verified features of the active PyTorch-model code.